

AgentOS — Comprehensive Project Report

Version: 1.0 · **Date:** June 14, 2026 · **Status:** Active Development **Tagline:** The Operating System for Enterprise AI Agents

1. Executive Summary

AgentOS is a deterministic, cost-efficient platform for building, deploying, managing, and monitoring autonomous AI agent workflows at enterprise scale. It replaces unstructured, prompt-heavy AI tooling with a visual, no-code Studio that lets users define agent teams, assign tasks, wire execution graphs, and export production-ready Python code — all from a browser.

The platform is built on three pillars:

- Deterministic Execution** — Workflows follow strict, user-defined state transitions (Flows) rather than letting LLMs decide the execution path, preventing infinite loops and unpredictable behavior.
- Cost Efficiency** — An intelligent RouterBrain dynamically selects the cheapest model capable of completing each task, combined with in-memory LLM response caching via LiteLLM.
- Full Observability** — A built-in "Flight Recorder" (TelemetryLogger) captures every prompt, response, tool call, token count, latency, and cost in real-time, giving developers a complete audit trail.

AgentOS is designed to be the base infrastructure layer upon which enterprises can build, run, and govern AI agents across departments — from sales and support to finance and engineering.

2. System Architecture

AgentOS employs a **three-tier architecture** with a clear separation of concerns:

Frontend (React + Vite) → Middleware (Node.js + Express) → Core Engine (Python)

2.1 Frontend (React + Vite)

FILE	PURPOSE
App.jsx	Root router — defines routes for Landing, Dashboard, Studio, Discovery, Governance
LandingPage.jsx	Marketing entry point with CTA to dashboard

FILE	PURPOSE
<code>Dashboard.jsx</code>	Control Plane — agent registry, metrics, flow execution, telemetry viewer
<code>Studio.jsx</code>	Visual drag-and-drop workflow builder (React Flow), AI blueprint generation, code export
<code>Discovery.jsx</code>	AI-powered use case generator from natural language business descriptions
<code>Governance.jsx</code>	RBAC user management, API key vault, audit log viewer
<code>CustomNodes.jsx</code>	Custom React Flow node types (Agent, Task, Trigger)
<code>Sidebar.jsx</code>	Collapsible navigation sidebar
<code>AgentContext.jsx</code>	Global state provider — manages agents, projects, flow execution, and telemetry

2.2 Middleware (Node.js + Express)

FILE	PURPOSE
<code>index.js</code>	Express server on port 3001 — all API endpoints and Python process spawning
<code>database.js</code>	Mock in-memory database with user wallet balance and credit deduction logic

2.3 Core Engine (Python)

FILE	PURPOSE
<code>agent.py</code>	<code>Agent</code> class — wraps LiteLLM for multi-provider LLM calls, in-memory caching, short-term memory
<code>task.py</code>	<code>Task</code> class — task execution with verification engine, smart routing, retry logic
<code>flow.py</code>	<code>Flow</code> class — deterministic sequential/async execution with max iteration limits
<code>router.py</code>	<code>RouterBrain</code> — AI-powered model selector that picks cheapest capable model per task
<code>telemetry.py</code>	<code>TelemetryLogger</code> — Flight Recorder; logs every prompt, response, tool call, tokens, latency, cost
<code>manager.py</code>	<code>ManagerBrain</code> — hierarchical orchestration mode where an AI manager delegates tasks to agents
<code>connectors.py</code>	<code>IntegrationsHub</code> — stub connectors for Slack, GitHub, Jira
<code>security.py</code>	<code>SecurityManager</code> (RBAC + audit logs) and <code>SecretManager</code> (mock encrypted vault)
<code>blueprint_generator.py</code>	Generates agent/task workflow graphs from natural language prompts (AI or keyword-based fallback)
<code>compiler.py</code>	Converts visual Studio graphs into exportable Python code (agents.py, tasks.py, crew.py, YAML configs)

FILE	PURPOSE
<code>discovery_engine.py</code>	Takes business context and generates top-3 high-ROI automation use cases via LLM
<code>universal_flow.py</code>	Runtime engine for Studio — executes arbitrary node graphs with DAG dependency resolution, tool calls, collaborative roundtables, and wallet deduction

3. API Reference

All endpoints are served from the Express server on **port 3001**.

Data Management

METHOD	ENDPOINT	DESCRIPTION
GET	<code>/api/wallet</code>	Fetch mock user wallet balance
GET	<code>/api/projects</code>	List all saved workflow projects
POST	<code>/api/projects</code>	Create or update a project (upsert by ID)
GET	<code>/api/agents</code>	List all agents with aggregated metrics
POST	<code>/api/agents</code>	Deploy a new agent
DELETE	<code>/api/agents/:id</code>	Remove an agent
PUT	<code>/api/agents/:id/toggle</code>	Toggle agent status (active ↔ idle)

Python Integration

METHOD	ENDPOINT	PYTHON SCRIPT	DESCRIPTION
POST	<code>/api/flows/run</code>	<code>example_flow.py</code>	Run the hardcoded two-agent research flow
POST	<code>/api/flows/custom</code>	<code>universal_flow.py</code>	Execute any Studio-built workflow (deducts wallet)
POST	<code>/api/discovery</code>	<code>discovery_engine.py</code>	Generate AI use case recommendations
POST	<code>/api/blueprint/generate</code>	<code>blueprint_generator.py</code>	Generate or modify workflow graphs from prompts
POST	<code>/api/blueprint/export</code>	<code>compiler.py</code>	Export visual graph to Python code files

4. Technology Stack

Frontend

TECHNOLOGY	VERSION	PURPOSE
React	19.2.6	UI framework
Vite	8.0.12	Build tool & dev server
React Router DOM	7.17.0	Client-side routing
@xyflow/react	12.11.0	Visual node editor (Studio Canvas)
Lucide React	1.17.0	Icon library

Backend

TECHNOLOGY	PURPOSE
Express.js	REST API server
CORS	Cross-origin request handling
child_process (spawn)	Python process orchestration

Python Core

TECHNOLOGY	PURPOSE
LiteLLM	Universal LLM abstraction layer (50+ providers)
python-dotenv	Environment variable management
DuckDuckGo Search	Live web search tool for agents

Supported LLM Providers (via LiteLLM)

PROVIDER	MODELS	TIER
Google	Gemini 2.5 Flash, Gemini 2.5 Pro	Default
OpenAI	GPT-4o, GPT-4o Mini	Supported
Anthropic	Claude 3.5 Sonnet	Supported
DeepSeek	DeepSeek V3, DeepSeek Coder	Supported

PROVIDER	MODELS	TIER
Groq	Llama 3 (8B), Mixtral (8×7B)	Supported (Ultra-fast)
Ollama	Llama 3, Mistral (Local)	Supported (Free/Local)

5. Problems, Solutions & Implementation Status

The following 20 problems have been identified as the core feature roadmap for AgentOS. Each is mapped to its current implementation status.

Legend

- ✔ **Implemented** — Working in codebase today
- 🟡 **Partially Implemented** — Foundation exists, needs production hardening
- ✖ **Not Yet Implemented** — Planned for future development

Major Problems (1–8)

Problem 1 — Unpredictable Agent Behavior

The same task can produce different outputs on different runs.

SOLUTION	STATUS	IMPLEMENTATION
Verification Engine	✔ Implemented	<code>task.py</code> — Tasks accept <code>validation_rules</code> . A dedicated verifier LLM checks outputs against rules with up to 3 retries.
Workflow Replay	🟡 Partial	Deterministic <code>Flow</code> class enforces fixed execution order, but full state snapshot replay is not yet implemented.

Impact: Critical for enterprise trust. Verification engine prevents bad outputs from propagating downstream.

Problem 2 — High Cost

Many agents call expensive models repeatedly.

SOLUTION	STATUS	IMPLEMENTATION
Smart Model Routing	✅ Implemented	<code>router.py</code> — <code>RouterBrain</code> evaluates cognitive load and selects the cheapest model per task. Toggle in Studio UI.
LLM Response Caching	✅ Implemented	<code>agent.py</code> — <code>litellm.cache = litellm.Cache(type="local")</code> enables in-memory caching.
Intermediate Result Reuse	✅ Implemented	<code>universal_flow.py</code> — DAG-based execution passes upstream results as context to downstream tasks.
SaaS Wallet & Cost Tracking	✅ Implemented	<code>telemetry.py</code> — Per-token cost calculation with 3x platform markup. <code>database.js</code> — Wallet deduction after each flow run.

Impact: Potential 50–90% cost reduction. Smart routing alone can cut costs by 10x on simple tasks.

Problem 3 — Difficult Debugging

When an agent fails, it's hard to know why.

SOLUTION	STATUS	IMPLEMENTATION
Agent Flight Recorder	✅ Implemented	<code>telemetry.py</code> — Records every prompt (full + preview), every response, every tool call, model used, latency, token counts, and cost.
Visual Workflow Timeline	✅ Implemented	Studio's Flight Recorder panel + Dashboard's Flow Execution Telemetry modal. Shows expandable trace cards with color-coded events, latency, token/cost badges.

Impact: Transforms debugging from guesswork to forensic analysis. Every LLM interaction is fully auditable.

Problem 4 — Agent Hallucinations

Agents confidently generate incorrect information.

SOLUTION	STATUS	IMPLEMENTATION
Mandatory Source Verification	🟡 Partial	Verification engine in <code>task.py</code> can enforce custom rules. No automatic source-checking yet.
Confidence Scoring	❌ Not Implemented	Planned — LLM self-evaluation of output confidence.
Cross-Agent Validation	🟡 Partial	Collaborative roundtable in <code>universal_flow.py</code> allows multiple agents to critique and refine each other's outputs.

Impact: Reduces incorrect outputs. Full implementation requires external verification tools (see Section 7).

Problem 5 — Agent Loops

Agents can get stuck talking to each other forever.

SOLUTION	STATUS	IMPLEMENTATION
Maximum Iteration Limits	✅ Implemented	<code>flow.py</code> — <code>max_iterations</code> parameter with hard cutoff. Default: 10.
Tool Call Loop Limits	✅ Implemented	<code>universal_flow.py</code> — Single-agent tool loops capped at 4 iterations. Collaboration rounds capped at <code>len(agents) * 2</code> .
Goal Completion Scoring	❌ Not Implemented	Planned — automatic detection of when output satisfies expected output.

Impact: Eliminates runaway costs and infinite execution loops.

Problem 6 — Too Much Manual Prompting

Users spend hours writing prompts.

SOLUTION	STATUS	IMPLEMENTATION
AI-Generated Agent Templates	✅ Implemented	<code>blueprint_generator.py</code> — Takes natural language (e.g., "Build a stock research team") and generates complete agent/task graphs. Uses LLM when API key is present, keyword-based smart fallback otherwise.
Auto-Generated Workflows	✅ Implemented	Studio home page accepts plain English prompts → animates agent/task node creation on canvas.
Quick Suggestion Chips	✅ Implemented	Studio UI provides clickable prompt suggestions ("Summarize customer support tickets", "Triage GitHub issues", etc.)

Impact: Reduces workflow creation from hours to seconds. Users describe intent; the system builds the team.

Problem 7 — Weak Memory

Agents forget earlier work in the same session.

SOLUTION	STATUS	IMPLEMENTATION
Short-Term Memory	✅ Implemented	<code>agent.py</code> — <code>memory_store</code> keeps last 5 interactions as chat history.
Long-Term / Persistent Memory	❌ Not Implemented	Requires external tooling (Mem0 or vector database). See Section 7.
Knowledge Graph	❌ Not Implemented	Planned for future development.

Impact: Short-term memory provides session continuity. Long-term memory is critical for agents that need to recall information across runs.

Problem 8 — Lack of Governance

Large companies need auditing, access control, and compliance.

SOLUTION	STATUS	IMPLEMENTATION
RBAC Permission System	✅ Implemented	<code>security.py</code> — <code>SecurityManager</code> with Admin/Operator/Viewer roles and action-level permissions.
Audit Logs	✅ Implemented	<code>security.py</code> — Immutable audit log with timestamps, user, action, and GRANTED/DENIED status.
Secrets Vault	✅ Implemented	<code>security.py</code> — <code>SecretManager</code> with mock encryption. Frontend vault UI in <code>Governance.jsx</code> .
UI Governance Console	✅ Implemented	<code>Governance.jsx</code> — Tabs for Users & Roles, Secrets Vault, and Audit Logs.

Impact: Foundation for enterprise compliance. Production deployment requires integration with real auth (OAuth/SAML) and encryption (AWS KMS / HashiCorp Vault).

Medium-Sized Problems (9–14)

Problem 9 — Slow Execution

SOLUTION	STATUS	IMPLEMENTATION
Parallel Agent Execution	✅ Implemented	<code>flow.py</code> — <code>run_async()</code> uses <code>asyncio.gather()</code> . <code>universal_flow.py</code> — Full DAG-based async execution with dependency resolution.

Problem 10 — Poor Tool Integration

SOLUTION	STATUS	IMPLEMENTATION
Web Search (DuckDuckGo)	✅ Implemented	<code>universal_flow.py</code> — Agents can call <code>[TOOL: WEB_SEARCH: query]</code> for live search.
Slack Connector	🟡 Stub	<code>connectors.py</code> — Stub implementation, prints to console.
GitHub Connector	🟡 Stub	<code>connectors.py</code> — Stub for issue creation.
Jira Connector	🟡 Stub	<code>connectors.py</code> — Stub for ticket creation.
Gmail, Notion, Databases	❌ Not Implemented	Planned for one-click connector marketplace.

Problem 11 — No Agent Performance Metrics

SOLUTION	STATUS	IMPLEMENTATION
Per-Agent Metrics	✅ Implemented	<code>telemetry.py</code> — Tracks accuracy (via verification), cost, latency, and token usage per agent per call.
Dashboard Aggregation	✅ Implemented	<code>Dashboard.jsx</code> — Displays total active agents, estimated daily cost, avg response time, system status.
Success Rate Tracking	❌ Not Implemented	Planned — historical success/failure rate per agent.

Problem 12 — Context Window Limits

SOLUTION	STATUS	IMPLEMENTATION
Memory Window Truncation	🟡 Partial	<code>agent.py</code> — Only injects last 5 memory items to stay within limits.
Automatic Context Compression	❌ Not Implemented	Planned — intelligent summarization of long contexts.
RAG-based Retrieval	❌ Not Implemented	Planned — retrieve only relevant chunks from large documents.

Problem 13 — Hard Deployment

SOLUTION	STATUS	IMPLEMENTATION
Code Export	✅ Implemented	<code>compiler.py</code> — Exports visual graphs to production-ready Python files (crew.py, agents.py, tasks.py, YAML configs, requirements.txt).
One-Click Cloud Deployment	❌ Not Implemented	Planned — Docker containerization + cloud deploy integration.

Problem 14 — Vendor Lock-in

SOLUTION	STATUS	IMPLEMENTATION
Multi-Provider Support	✅ Implemented	Built on LiteLLM, supporting 50+ providers. Studio UI offers model selection dropdown with Google, OpenAI, Anthropic, DeepSeek, Groq, and local Ollama models.
Seamless Model Switching	✅ Implemented	Change model per agent without code changes — just select from dropdown in properties panel.

Minor Problems (15–20)

#	PROBLEM	SOLUTION	STATUS
15	Prompt Versioning	Git-like version tracking for prompts	❌ Not Implemented
16	Poor Collaboration	Simultaneous multi-developer workflow editing	❌ Not Implemented
17	No Workflow Marketplace	Marketplace for pre-built agent templates (trading, marketing, coding, research)	❌ Not Implemented
18	Weak Security	Encryption, secret management, RBAC	✅ Implemented (foundation)
19	Difficult Monitoring	Real-time dashboards	✅ Implemented
20	No Cost Forecasting	Pre-execution cost estimates ("This workflow will cost ~₹15/day")	❌ Not Implemented

6. Implementation Status Summary

CATEGORY	IMPLEMENTED	PARTIAL	NOT YET	TOTAL
Major Problems (1-8)	5	2	1	8
Medium Problems (9-14)	4	1	1	6
Minor Problems (15-20)	2	0	4	6
Totals	11	3	6	20

~70% of the identified problem areas have at least foundational implementations.

7. External Tools & Services Required

The following external tools/services are required to bring AgentOS to production-grade enterprise quality. These cannot be replicated with custom code alone and represent the "buy vs. build" decisions.

CAPABILITY	RECOMMENDED TOOL	ESTIMATED MONTHLY COST	PURPOSE	INTEGRATION PRIORITY
Memory Persistence	Mem0	\$100 – \$500/mo	Long-term agent memory across sessions. Replaces in-memory <code>memory_store</code> . Enables agents to recall past interactions, user preferences, and learned patterns.	🔴 High
Observability & Tracing	LangFuse / SigNoz	Free – \$500/mo	Production-grade tracing, dashboards, and alerting. Extends the built-in Flight Recorder with persistent storage, team-wide dashboards, and anomaly detection.	🔴 High
Output Verification	Patronus / Arize	\$500+/mo	Automated hallucination detection, factual accuracy scoring, and output quality evaluation at scale. Replaces basic rule-based verification in <code>task.py</code> .	🟡 Medium
Cost Optimization	TrueFoundry / Custom Layer	\$200 – \$1,000/mo	Advanced model routing with historical performance data, batch processing, and request queuing for cost reduction beyond <code>RouterBrain</code> .	🟡 Medium

CAPABILITY	RECOMMENDED TOOL	ESTIMATED MONTHLY COST	PURPOSE	INTEGRATION PRIORITY
A/B Testing	LangFuse / Custom Script	Free – \$300/mo	Compare agent performance across different models, prompts, and configurations. Statistical significance testing on output quality.	● Low
Compliance & Audit	Enterprise AMP Platform	\$250 – \$2,000+/mo	SOC 2 / ISO 27001 audit trails, compliance dashboards, data residency controls, and enterprise SSO integration.	● Medium

Total Estimated External Tooling Cost

TIER	MONTHLY ESTIMATE
Minimum Viable (Open-source where possible)	\$300 – \$800/mo
Standard Enterprise	\$1,500 – \$3,500/mo
Full Enterprise Suite	\$3,000 – \$5,000+/mo

8. Key Gaps by Pricing Tier

Free Tier — Currently Missing

GAP	SEVERITY	IMPACT
✗ Discovery Feature (should be paid-only)	● Medium	Discovery Hub currently accessible to all users. Needs gating behind paid tier.
✗ Observability Dashboard	● Critical	Flight Recorder exists but is ephemeral (in-memory). No persistent dashboards or historical analytics.
✗ Governance / RBAC enforcement	● Critical	<code>SecurityManager</code> exists in Python but is not enforced at the API layer. Any user can access any endpoint.
✗ Hallucination Verification	● Medium	Verification engine exists but is opt-in (requires <code>validation_rules</code>). No automatic hallucination detection.
✗ Enterprise Support	● Low	No SLA, priority support channel, or dedicated account management.

Enterprise Tier — Still Missing

GAP	SEVERITY	IMPACT
✗ Cost Optimization Routing (Production-Grade)	● Critical	<code>RouterBrain</code> works but lacks historical performance data, A/B testing, and batch optimization.
✗ Native Memory Persistence	● Critical	Only in-memory short-term (5 messages). No cross-session, cross-workflow, or team-shared memory.
✗ Context Window Auto-Management	● Medium	Basic truncation only (last 5 memories). No intelligent summarization, RAG, or sliding window compression.
✗ SOC 2 / ISO 27001 Certification	● Critical	Platform is not certified. Enterprise customers in regulated industries (finance, healthcare) will require this.
✗ Real-Time Collaboration	● Medium	No WebSocket-based live editing. Only one user can edit a workflow at a time.
✗ Workflow Marketplace	● Low	No ability to share, discover, or monetize pre-built agent templates.

9. Codebase Structure

```

AgentOs/
├── .env                # Root environment variables
├── .gitignore
├── README.md
├── package.json        # Frontend dependencies & scripts
├── vite.config.js      # Vite configuration
├── index.html          # Entry HTML
├──
├── src/               # Frontend React Application
│   ├── App.jsx         # Router & route definitions
│   ├── App.css         # App-level styles
│   ├── index.css       # Global design system
│   ├── main.jsx        # React DOM entry point
│   ├── pages/
│   │   ├── LandingPage.jsx # Marketing landing page
│   │   ├── Dashboard.jsx  # Control Plane (agents, metrics, flow runner)
│   │   ├── Studio.jsx     # Visual workflow builder (825 lines)
│   │   ├── Discovery.jsx  # AI use case generator
│   │   └── Governance.jsx  # RBAC, secrets, audit logs
│   ├── components/
│   │   ├── CustomNodes.jsx # React Flow node definitions
│   │   ├── DashboardLayout.jsx # Layout wrapper with sidebar
│   │   └── Sidebar.jsx     # Navigation sidebar
│   ├── context/
│   │   └── AgentContext.jsx # Global state management
│   └── assets/
├──
├── server/            # Node.js Express API
│   └── index.js        # API endpoints (319 lines)

```

```
| ├── database.js           # Mock in-memory database
| ├── package.json         # Server dependencies
| └── node_modules/
└── core/                  # Python Core Engine
    ├── setup.py           # Python package definition (agentos v0.1.0)
    ├── .env               # API keys
    ├── .env.example       # Template for API keys
    ├── example_flow.py    # Hardcoded 2-agent research demo
    ├── universal_flow.py  # Runtime: executes any Studio-built graph
    ├── discovery_engine.py # AI use case generator
    ├── compiler.py        # Graph → Python code exporter
    └── agentos/           # Core Python package
        ├── __init__.py    # Exports: Agent, Task, Flow, RouterBrain
        ├── agent.py       # Agent class (LiteLLM, caching, memory)
        ├── task.py        # Task class (verification, routing, retries)
        ├── flow.py        # Flow class (sequential + async execution)
        ├── router.py      # RouterBrain (smart model selection)
        ├── telemetry.py   # TelemetryLogger (Flight Recorder)
        ├── manager.py     # ManagerBrain (hierarchical orchestration)
        ├── connectors.py  # IntegrationsHub (Slack, GitHub, Jira stubs)
        ├── security.py    # SecurityManager + SecretManager
        └── blueprint_generator.py # AI workflow graph generator
```

10. Key Differentiators

FEATURE	AGENTOS	TYPICAL COMPETITORS
Execution Model	Deterministic Flows with explicit state transitions	LLM-decided execution paths (unpredictable)
Cost Control	Built-in RouterBrain + per-token cost tracking + wallet system	Manual model selection, no cost awareness
Visual Builder	Full drag-and-drop Studio with AI generation + code export	CLI-only or basic YAML configs
Observability	Flight Recorder with full prompt/response/tool/cost logging	Minimal or no built-in tracing
Multi-Provider	50+ LLM providers via LiteLLM (including local models)	Often locked to single provider
Monetization	Built-in SaaS markup (3x cost) with wallet/credits system	No built-in monetization layer
Code Export	One-click export to production Python code	No export capability

11. Product Roadmap (Prioritized)

Phase 1 — Foundation Hardening (Current → Q3 2026)

- ☐ Enforce RBAC at API layer (middleware auth)
- ☐ Persistent database (PostgreSQL / MongoDB)
- ☐ WebSocket-based real-time execution streaming
- ☐ Persistent telemetry storage
- ☐ Pre-execution cost forecasting

Phase 2 — Enterprise Features (Q3 → Q4 2026)

- ☐ Mem0 integration for persistent agent memory
- ☐ LangFuse / SigNoz integration for observability
- ☐ Prompt versioning system
- ☐ OAuth/SAML SSO integration
- ☐ Docker containerization + one-click deployment

Phase 3 — Marketplace & Scale (Q1 → Q2 2027)

- ☐ Workflow marketplace (publish, discover, fork templates)
- ☐ Real-time multi-user collaboration (CRDT-based)
- ☐ Advanced context management (RAG + auto-compression)
- ☐ Full connector marketplace (Gmail, Notion, databases)
- ☐ SOC 2 / ISO 27001 certification process

Phase 4 — Intelligence Layer (Q2 2027+)

- ☐ Patronus/Arize integration for hallucination verification
 - ☐ Confidence scoring on all agent outputs
 - ☐ Automated A/B testing for agent configurations
 - ☐ Goal completion detection (auto-stop when output meets criteria)
 - ☐ Advanced cost optimization with historical routing data
-

12. How to Run

Frontend (Vite Dev Server)

```
cd /Users/parthlodaya/Downloads/AgentOs  
npm install
```

```
npm run dev
```

Backend (Express API Server)

```
cd /Users/parthlodaya/Downloads/Agent0s/server  
npm install  
node index.js
```

Python Core (Install Package)

```
cd /Users/parthlodaya/Downloads/Agent0s/core  
pip install -e .  
pip install litellm python-dotenv duckduckgo-search
```

Environment Variables

Copy `.env.example` in `/core/` and add your API keys:

```
GEMINI_API_KEY=your_key_here  
OPENAI_API_KEY=your_key_here
```

Note: The platform works without API keys using mock/fallback responses for testing.